# Getting data and scaling up

## Programming & Data Management, Part II (Python) · Session 4

Magnus Lodefalk
*Örebro University*

NA446A, Örebro University

Autumn 2026



# Today's goals

By the end of today you can:

1. Fetch data from an open API, save the raw answer, and check that every row arrived
2. Handle dates: months to quarters with a stated rule, and growth; do a repeated step with a function and a loop over files
3. Rebuild everything with one script that logs rows in and out; query a file with DuckDB; end with one regression table

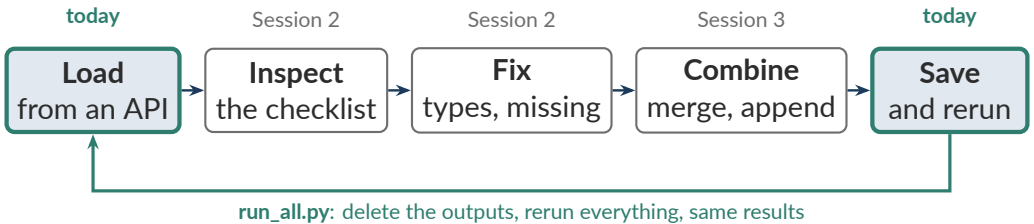
# From last time

Answer on your own first, then compare with a neighbour

1. You merge workers onto firms with `validate="m:1"` and pandas stops with a `MergeError`. What does that tell you?
2. Write the line that gives each person's income in the year before, in a panel sorted by person and year.

# Every data task follows one recipe

Today: load from an API, then save and rerun (everything)




- Today's two errors never crash a script: an API answer that stops after the first page, and a quarter built from two months.

# Seven terms for today

| | |
|---|---|
| API | a web address that a program asks for data; it answers in a fixed format |
| JSON | the text format of most answers: lists `[ ]` and dictionaries `{ }` |
| metadata | what a table holds (variables, months); ask for it before the data |
| raw answer | the response saved exactly as it came, in `data/raw/` |
| `"QE"` | pandas' code for "quarter, labelled by its end date" (month: `"ME"`) |
| function | named lines with inputs and a result: `def f(x): ... return ...` |
| `run_all.py` | one script that runs every step in order and logs rows in and out |

- An API answer is raw data: save it first, then work on the copy.

# Steps 1–6: from SCB's answer to inflation

| | Question | Tool (Stata) |
|---|---|---|
| 1 | what does the table hold? | `requests.get`, `.json()` (none built in) |
| 2 | how do we fetch the data, and keep a copy? | `requests.post`, `write_text` (`copy`) |
| 3 | did every month arrive? | `assert` on the count (`assert _N == ...`) |
| 4 | how do month labels become dates? | `to_datetime` (`monthly()`) |
| 5 | how do months become quarters? | `resample("QE")`, `count` (`collapse`) |
| 6 | how much did prices rise over the year? | `pct_change(4)` (`L4.`) |

- Every step ends with a count: how many rows the source says exist, and how many we have.

# Step 1: what does the table hold?

```
r = requests.get(URL, timeout=30)
r.raise_for_status()
meta = r.json()
months = meta["variables"][1]["values"]
print(len(months), months[0], months[-1])
```

```
560 1980M01 2026M08
```

`URL`: SCB's address for the consumer price index, one value per month.
`raise_for_status()` stops the script if the server answered with an error, instead of carrying on with an empty answer.

- Before any data: SCB says the table holds 560 months, January 1980 to August 2026. Remember the 560.

`.json()`: the text as lists and dictionaries; `["variables"][1]`: the second variable, time. No key needed.

# Step 2: how do we fetch the data, and keep a copy?

```
r = requests.post(URL, json=QUERY, timeout=30)
r.raise_for_status()
(RAW / "cpi.json").write_text(r.text)
rows = r.json()["data"]
print(rows[0], rows[-1], sep="\n")
```

```
{'key': ['1980M01'], 'values': ['28.38']}
{'key': ['2026M08'], 'values': ['124.85']}
```

`post` sends a question with the request (`QUERY`: this series, all months). `write_text` saves the answer as a file. The month is a label and the value is text, in quotes.

- The raw answer is saved before anything else: SCB revises its numbers and changes the base year, and one day the address may change.

# Step 3: did every month arrive?

```
print(len(rows), len(months))
assert len(rows) == len(months), "months missing"
```

```
560 560
```

`assert` turns the check into a rule: if the two numbers ever differ, the script stops with “months missing”.

- Compare what you got with what the source says exists. This is the check people skip, because a short answer raises no error.

Stata: `assert _N == 560`.

# World Bank, page 1: which countries are on it?

Your bet first: hands up. The answer is on the next slide.

A. All five, about ten rows each
B. All five, the newest years only
C. One country only

# Does page 1 hold everything? The World Bank

```
r = requests.get(WB, params={"format": "json"})
meta_wb, page1 = r.json()
print(meta_wb["total"], meta_wb["pages"])
print(len(page1), page1[0]["countryiso3code"],
      page1[-1]["countryiso3code"])
```

```
330 7
50 DNK DNK
```

`WB`: GDP per person for Sweden, Finland, Norway, Denmark and Iceland, 1960–2025. The answer is a list: first the metadata, then one page of rows.

- 330 rows exist, in 7 pages. Page 1 holds 50, and all 50 are Denmark. A script that reads only page 1 raises no error.

# Step 4: how do month labels become dates?

```
cpi = pd.DataFrame(rows)
month = cpi["key"].str[-1]
cpi["date"] = pd.to_datetime(month, format="%YM%m")
cpi["cpi"] = pd.to_numeric(cpi["values"].str[0])
cpi[["date", "cpi"]].tail(2)
```

```
          date     cpi
558 2026-07-01  125.17
559 2026-08-01  124.85
```

`str[-1]`: the last part of each key, the month. Format: year (`%Y`), the letter M, month (`%m`). `to_numeric`: text to numbers.

- Now pandas knows these are dates (Stata: `monthly(month, "YM")`).

# Step 5: how do months become quarters?

```
cpi = cpi.set_index("date")["cpi"]
q = cpi.resample("QE").mean()
n = cpi.resample("QE").count()
pd.DataFrame({"cpi": q, "months": n}).tail(3).round(2)
```

| date | cpi | months |
|---|---|---|
| 2026-03-31 | 124.84 | 3 |
| 2026-06-30 | 124.84 | 3 |
| 2026-09-30 | 125.01 | 2 |

- The rule is economics: an index or a rate takes the mean, a flow (job ads) the sum. And the latest quarter has only 2 months so far.

`set_index`: dates as row labels; `resample("QE")`: by quarter; `count()`: months with a value.

# Step 6: how much did prices rise over the year?

```
q = q[n == 3]
infl = q.pct_change(4) * 100
infl.tail(3).round(1)
```

```
date
2025-12-31    0.5
2026-03-31    0.5
2026-06-30    0.4
Freq: QE-DEC, Name: cpi, dtype: float64
```

`q[n == 3]`: complete quarters only. `pct_change(4)`: each quarter against the one four rows earlier, the same quarter a year before, so seasons cancel.

- Prices rose 0.4 per cent over the year to the second quarter of 2026.

Stata, after `tsset q`: `gen infl = 100*(cpi/L4.cpi - 1)`.

# Check your understanding

Answer on your own first, then compare with a neighbour

Predict: (a) you turn monthly *job ads* into quarters with `resample("QE").mean()`. What is wrong? (b) An old answer online writes `resample("Q")`. What happens in pandas 3?

# Steps 7–10: from one file to one table

| | Question | Tool (Stata) |
|---|---|---|
| 7 | how do we write a repeated step once? | `def ... return` (`program define`) |
| 8 | how do we read every file the same way? | a `for` loop, `glob` (`foreach`) |
| | how do we rebuild everything, and check it? | `run_all.py`, a log (a master do-file) |
| | how do we query a file too large to load? | DuckDB, SQL (none built in) |
| 9 | does inflation move with unemployment? | `smf.ols`, HAC (`newey`) |
| 10 | how do we save the table for a paper? | `summary_col` (`esttab`) |

- Every step answers one question. Say the question before you write the code.

# Step 7: how do we write a repeated step once?

```
def read_scb(path):
    d = pd.DataFrame(json.loads(path.read_text())["data"])
    month = d["key"].str[-1]
    d.index = pd.to_datetime(month, format="%YM%m")
    return pd.to_numeric(d["values"].str[0])
```

Steps 2 and 4 in one place: read one saved answer, return a monthly series with dates as its index. `path` is the input; `return` hands back the result. Running the cell prints nothing: it only *defines* the function.

- Write a step once, name it, and test it. A fix in the function fixes every file that uses it.

`json.loads`: text to lists and dictionaries. Needs `import json`. The lab's version adds `encoding="utf-8"` to `read_text`, for Windows. Stata: `program define`.

# Step 8: how do we read every file the same way?

```
series = {}
for path in sorted(RAW.glob("*.json")):
    series[path.stem] = read_scb(path)
monthly = pd.concat(series, axis=1)
monthly.tail(2)
```

```
              cpi  unemp
key
2026-07-01  125.17    8.6
2026-08-01  124.85    8.9
```

`glob`: every JSON file; `stem`: the name without `.json`; `axis=1`: one column per file.

- Two files today, fifty tomorrow: the loop does not change.

# How do we rebuild everything? `run_all.py`

```
SCRIPTS = [
    "scripts/01_get_data.py",
    "scripts/02_clean.py",
    "scripts/03_table.py",
]
```

```
$ uv run run_all.py
01_get_data.py: cpi 560/560 months, unemp 308/308 months (SCB API)
02_clean.py: 868 monthly values in, 102 quarters out (2001Q1-2026Q2)
  dropped 84 of 186 quarters with a gap (84 before the unemployment data)
03_table.py: 102 quarters in, 2 models out (output/tables/regression.*)
run_all done: log in output/logs/run_log.txt
```

- The list is the recipe; each script prints rows in, out, and why.

# The delete-outputs test

Gentzkow and Shapiro (2014), *Code and Data for the Social Sciences*, ch. 2

1. Delete `data/processed/` and `output/`: everything the code made
2. Run `run_all.py` once
3. Everything comes back, with the same numbers; the log in `output/logs/run_log.txt` shows rows in and out of every step

   *"The proof in the pudding is that we can now delete all of the output files in the directory [...] and reproduce them by running `rundirectory.bat`. This is the precise sense in which the output is now replicable."*

– The log is the first thing to read when a number looks strange: a step that drops half the sample shows there before it shows in the regression.

▷ The same pipeline in Stata

# DuckDB: how do we query a file too large to load?

```
duckdb.sql("""
  SELECT year(date) AS year, AVG(infl) AS infl
  FROM 'data/processed/quarterly.parquet'
  WHERE year(date) >= 2024
  GROUP BY year ORDER BY year""").df().round(2)
```

| | year | infl |
|---|---|---|
| 0 | 2024 | 2.86 |
| 1 | 2025 | 0.68 |
| 2 | 2026 | 0.47 |

SQL: the language of databases. Parquet: a compact file format for tables.

- DuckDB reads only the rows and columns it needs, so it works on files too large for memory. `.df()` gives a DataFrame.

# Step 9: does inflation move with unemployment?

```
q = pd.read_parquet("data/processed/quarterly.parquet")
m1 = smf.ols("infl ~ unemp", data=q).fit(
    cov_type="HAC", cov_kwds={"maxlags": 4})
m2 = smf.ols("infl ~ unemp + infl_lag1", data=q).fit(
    cov_type="HAC", cov_kwds={"maxlags": 4})
```

A Phillips-curve regression: quarterly inflation on the unemployment rate, 2001–2026; model 2 adds last quarter's inflation. HAC (Newey–West) standard errors allow errors that are correlated over time, with 4 quarters of lags.

- The regression reads the file the pipeline saved, never numbers typed by hand.

`"infl ~ unemp"`: inflation on unemployment, constant added. Stata: `newey infl unemp, lag(4)` gives the same coefficients.

# Step 10: how do we save the table for a paper?

```
t = summary_col([m1, m2], stars=True,
                float_format="%.2f")
t.tables[0].loc["N"] = [str(int(m1.nobs)), str(int(m2.nobs))]
t.tables[0].to_csv(TABLES / "regression.csv")
(TABLES / "regression.tex").write_text(t.as_latex())
```

```
607
```

`summary_col` puts models side by side: coefficients with stars, standard errors in parentheses. The third line adds a row with each model's N (as text, so it prints 102, not 102.0000). The 607 is `write_text` reporting the characters it wrote.

- The CSV opens in Excel and Word; the `.tex` goes straight into a paper. Never copy numbers from the screen.

Needs `from statsmodels.iolib.summary2 import summary_col`. Stata: `esttab` (from SSC).

# The table, as the pipeline saved it

```
================================
                 infl I  infl II
--------------------------------
Intercept       6.46***  0.98
                (1.35)   (0.79)
unemp           -0.61*** -0.11
                (0.18)   (0.10)
infl_lag1                0.93***
                         (0.09)
R-squared       0.06     0.88
R-squared Adj.  0.05     0.88
N               102      102
================================
```

- Model I: a quarter with one point higher unemployment had 0.6 points lower inflation, on average
- Model II: most of that is last quarter's inflation carrying over (0.93); the unemployment coefficient falls to $-0.11$
- A correlation over time, not a causal effect

- N = 102 is the number in the log. The table and the log must agree.

▷ Rerun next month

# For the exam period: predict, test, explain

1. Predict before you run: write down what the line prints. The exam asks exactly that, in WISEflow, without Python and without AI
2. Test on a case whose answer you know: count rows in and out, `assert` what must hold, compute one number by hand
3. Explain every line in one sentence, including the lines an AI wrote. If you cannot, you do not own the result yet

– Code that runs is not code that is right. Code you can predict, test and explain is.

Practise: the sample exam and the practice bank on the course site, without AI.

# Today's lab: a mini-pipeline in pairs

1. Pilot and co-pilot: one types, one reads the slide and predicts; swap at each script. Fill in TODOs 1–5 in `01_get_data.py`, `02_clean.py`, `03_table.py`, `run_all.py` *goals 1–3*
2. Run `check_s4.py`: it does the delete-outputs test for you *goal 3*
3. Review an AI's pipeline: `ai_02_clean.py` runs without an error. Write three `asserts` that catch it, then fix it *rules 3–4*

– No network, or SCB answers 429 ("too many requests")? Set `USE_SAVED = True`: the saved copy of 29 September runs the same way.

Never paste personal or register data into a chatbot. API keys (none today) go in `.env`, never in code.

# Summary

1. An API: ask for the metadata, ask for the data, save the raw answer, `assert` the count; an answer in pages needs a loop to the last page
2. Dates: `to_datetime` with a format; `resample("QE")` with a rule and a count; `pct_change(4)`; one function, one loop over the files
3. `run_all.py` rebuilds everything and logs rows in and out; DuckDB queries a saved file; `summary_col` gives one table as CSV and LaTeX

**Before you go (2 minutes):** three anonymous questions on Part II, on the course site.
**Before the exam:** the World Bank exercise and its self-check (about 90 minutes; solution after three days), then the sample exam, without AI.

# Test yourself before next time

Without AI: this is what you should manage on your own

1. The metadata lists 308 months; your data have 300 rows. Name two likely causes and the line that stops your script.
2. Write the line that turns labels such as `"2024M07"` into dates.
3. Monthly hours worked to quarters: which rule, and why? And for the unemployment rate?
4. A function lacks `return`: what does it give back, and `pd.concat` then?
5. Write the SQL that gives the mean of `infl` per year from `quarterly.parquet`.
6. What is the delete-outputs test, and what does it prove?
7. A script reads only the World Bank's page 1. What does it report, and what stops it?

# Appendix

# The question SCB receives (QUERY)

```
QUERY = {"query": [
    {"code": "ContentsCode",
     "selection": {"filter": "item", "values": ["00000807"]}},
    {"code": "Tid",
     "selection": {"filter": "all", "values": ["*"]}}],
    "response": {"format": "json"}}
```

- One entry per variable of the table: here the series (`00000807`, the CPI since 1980) and time (`Tid`, all months: `"*"`)
- A table with more variables needs one entry each: the lab's unemployment rate picks labour status, seasonal adjustment, sex and age
- The lab's `01_get_data.py` builds the query in a loop

# Every page, with a loop (the exercise)

```
meta, rows = get_page(name, 1)
for page in range(2, meta["pages"] + 1):
    rows = rows + get_page(name, page)[1]
assert len(rows) == meta["total"], "rows missing"
```

- Page 1 says how many pages exist; the loop asks for the rest
- `get_page` (given in the exercise) sends `page=...` with the request and saves each page in `data/raw/wb/`
- Last year's course code set `per_page=130` for 132 rows and silently lost two years of Swedish data: a larger page is not a check

# The same pipeline in Stata (run on the lab's data)

| pandas | Stata |
|---|---|
| `pd.to_datetime(month, format="%YM%m")` | `gen m = monthly(month, "YM")` |
| `resample("QE").mean(), .count()` | `collapse (mean) cpi (count) n=cpi, by(q)` |
| `pct_change(4) * 100` | `tsset q` then `gen infl = 100*(cpi/L4.cpi - 1)` |
| `shift(1)` | `gen infl_lag1 = L.infl` |
| `def read_scb(path): ...` | `program define` |
| `for path in sorted(RAW.glob("*.json")):` | `foreach f of local files { ... }` |
| `run_all.py` with a log | a master do-file with `log using` |
| `fit(cov_type="HAC", ...)` | `newey infl unemp, lag(4)` |
| `summary_col, as_latex()` | `esttab` (from SSC) |

Same 102 quarters and the same coefficients: $-0.61$ (0.18); with the lag $-0.11$ and 0.93. The intercept's standard error differs in the second decimal (1.35 vs 1.37): statsmodels applies a small-sample correction.

# API keys go in `.env`, never in code

```
from dotenv import load_dotenv
import os
load_dotenv()                    # reads .env
key = os.getenv("FRED_API_KEY")
```

- `.env` is a text file in the project folder with one line: `FRED_API_KEY=...`
- `.gitignore` lists `.env`, so the key never reaches GitHub
- SCB and the World Bank need no key; FRED (US data) does

# Rerun next month: what changes?

- After SCB publishes September for both series, the third quarter of 2026 has three months: N rises from 102 to 103, and every coefficient moves a little
- The raw answers change too: SCB revises its numbers and changes the base year
- To reproduce today's table later, keep today's saved answers: `USE_SAVED = True` uses the copy of 29 September
- A pipeline that reruns is not a result that stays: the saved raw answer is what makes a table reproducible.